ME 656: Autonomous Navigation for Mobile Robots

Simulation Lab 2 Report

Course:	ME 656 - Autonomous Navigation for Mobile Robots
Semester:	Spring 2026
Assignment:	SimLab 2
Student:	Matthew Lepis

1 Overview

This report summarizes the path-planning and localization-uncertainty results for Simulation Lab 2. The robot was planned through the provided 2D obstacle workspace using a Rapidly-exploring Random Tree (RRT). For each generated path, a linear Kalman filter covariance propagation was used to estimate the expected localization uncertainty along the path.

The final run used 1000 RRT trials. From those trials, four representative paths were selected: the shortest path, the minimum terminal uncertainty path, the maximum terminal uncertainty path, and a best-tradeoff path. The best-tradeoff path was selected using a normalized cost function with equal weighting between path length and terminal uncertainty.

2 Selected Paths

Path type	Trial	Path length (m)	Terminal uncertainty, $\mathrm{tr}(P)$
Shortest path	193	100.00	47.42
Minimum uncertainty	442	108.00	9.56
Maximum uncertainty	752	130.00	251.39
Best tradeoff	811	103.10	13.55

Table 1: Selected path outcomes from 1000 RRT trials.

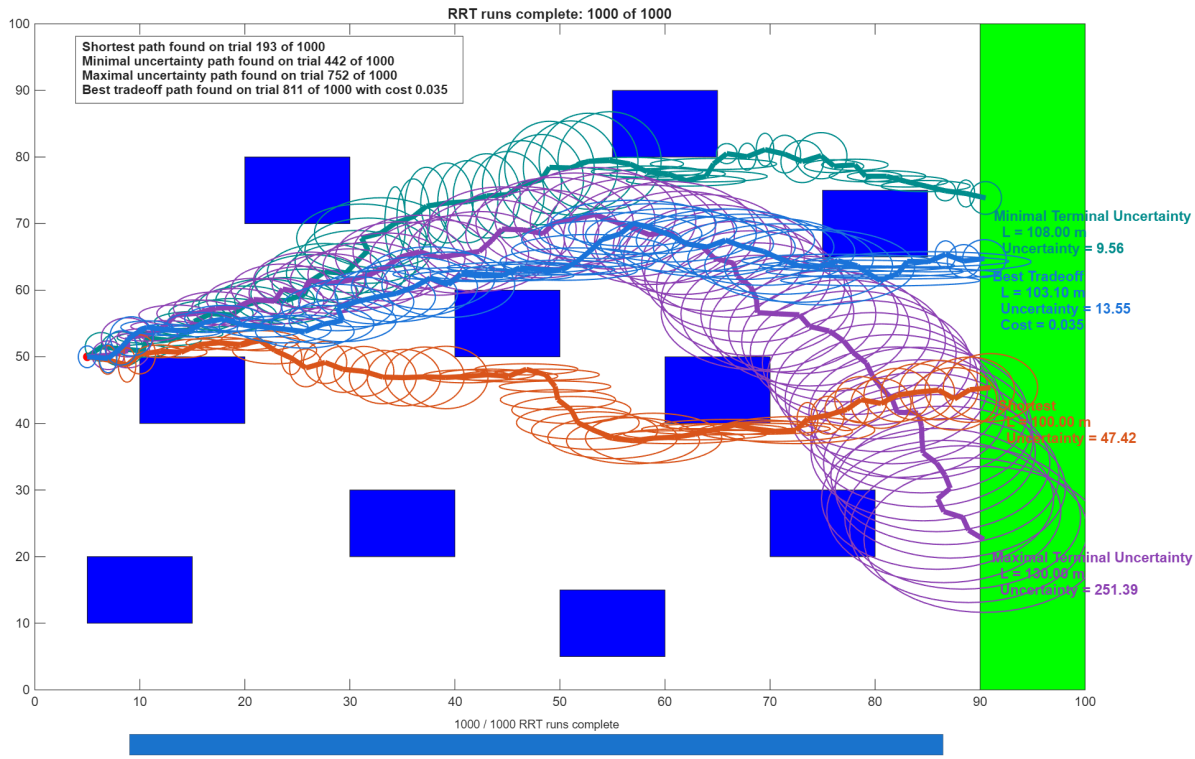

Figure 1: Selected RRT paths and Kalman filter covariance ellipses for the 1000-trial run.

The shortest path is the most direct route to the goal. It moves nearly horizontally across the workspace while avoiding obstacles along the way. This keeps the total distance low at 100 m, but the terminal uncertainty is higher than both the minimum-uncertainty and best-tradeoff paths. The shortest and minimum-uncertainty paths pass comparably near obstacles, but they do so at different points in the workspace.

The minimum-uncertainty path takes a longer route, with a path length of 108 m, but ends with much lower terminal uncertainty. A key difference is that near the end of its path, the robot observes an obstacle closer to the goal than it does on the shortest path. This gives the Kalman filter additional position information later in the trajectory, which reduces the final covariance.

The maximum-uncertainty path is both long and poorly localized. It appears to spend the longest duration outside useful sensing range, especially toward the end of the path. During these portions of the trajectory, process noise continues to increase the covariance while there are few or no position measurements to reduce it. As a result, this path reaches a terminal uncertainty of 251.39.

3 Distance, Uncertainty, and Collision Risk

Traveling near obstacles can improve localization because the robot can compare range-beam observations against the known map. When an obstacle is visible from a horizontal beam, the robot effectively gains information about its x-position. When an obstacle is visible from a vertical beam, it gains information about its y-position. These measurements reduce the corresponding components of the error covariance matrix.

This same behavior also introduces collision risk. Because the robot has nonzero localization uncertainty, the actual robot position may differ from the nominal planned state. If a covariance

ellipse intersects an obstacle, then there is a plausible localization error under which the robot could be in collision even though the nominal path is collision-free. Therefore, a good path should not simply stay as close as possible to obstacles. It should use obstacle visibility enough to reduce uncertainty while still maintaining reasonable clearance.

Additional distance is worthwhile only when it produces a meaningful reduction in uncertainty. A longer path that does not pass through informative sensing regions can increase distance without improving localization. To capture this tradeoff, the best-tradeoff path was selected using

$$J = w_d \bar{d} + w_u \bar{u},$$

where $\bar{d}$ is normalized path length, $\bar{u}$ is normalized terminal uncertainty, and $w_d = w_u = 0.5$ for this run. With equal weighting, the best-tradeoff path had a path length of 103.1 m and terminal uncertainty of 13.55. This is slightly longer than the shortest path, but it substantially reduces terminal uncertainty. It is also less certain than the minimum-uncertainty path, but it achieves most of the uncertainty reduction with less additional distance.

The most reasonable weighting depends on the application. A time-critical task would place more weight on distance, while a precision navigation task would place more weight on uncertainty. For this run, the equal-weighted best-tradeoff path is a useful middle ground between minimizing travel distance and minimizing terminal covariance.

4 Number of RRT Trials

Increasing the number of RRT trials improved the quality of the selected paths. With more trials, the algorithm had more opportunities to sample paths that were smoother, more direct, or more informative from a localization standpoint. In this workspace, 1000 trials produced noticeably better paths than 100 trials.

Based on these results, 100 trials is useful but not reliably sufficient for finding a high-quality path. More than 100 trials gives the selection process a better set of candidate paths, especially when optimizing for both path length and localization uncertainty. For this workspace, 1000 trials provided a more reliable basis for identifying the shortest, minimum-uncertainty, maximum-uncertainty, and best-tradeoff paths.

5 Assumptions

These observations are based on the SimLab 2 assumptions: the robot is modeled as a point, the workspace obstacles are known and rectangular, the robot follows the planned path exactly, and the Kalman filter is used to predict covariance growth rather than to correct a simulated perturbed trajectory. The results also depend on randomized RRT samples, so individual selected paths can change from run to run.

6 Included MATLAB Files

The MATLAB source associated with this report is organized as follows:

- `simlab2/simlab2.m`: driver script that runs the RRT trials, propagates covariance, selects representative paths, and plots results
- `simlab2/RRT.m`, `simlab2/Tree.m`, and `simlab2/TreeNode.m`: RRT implementation and tree data structures

- `simlab2/propagate_KF_path.m` and `simlab2/collect_sensor_data.m`: Kalman covariance propagation and obstacle-visibility measurement logic
- `simlab2/find_shortest_path.m` and `simlab2/find_best_path.m`: path-selection helpers
- `simlab2/plot_path.m`, `simlab2/plot_covariance_evolution.m`, and `simlab2/selection_color.m`: plotting utilities for selected paths and covariance ellipses
- `simlab2/generate_obstacles.m`, `simlab2/collision_check_point.m`, `simlab2/collision_check_segment.` and `simlab2/ellipse.m`: workspace, collision-checking, and ellipse helper functions

A MATLAB Source Code

`simlab2.m`

Listing 1: `simlab2.m`

```
%% simlab2.m
% Author: Matthew Lepis
% Course: ME 656 - Autonomous Navigation for Mobile Robots
% Assignment: SimLab 2
%
% File: simlab2.m
%
% Description:
%   Runs the SimLab 2 RRT and Kalman filter workflow, selects the shortest,
%   minimum-uncertainty, maximum-uncertainty, and best-tradeoff paths, and
%   plots the results.

%% Establish configuration constants & constraint
% "original"
% "maze"
% "spiral"
cfg = simulation_configuation("original", 1000, 33);

%% Run 'Rapidly expanding Random Tree' (RRT) trials to find many valid paths to goal.
RRT_trials = RRT(cfg);
% RRT_trials{k}.path - holds the path array of all `TreeNodes` in `k` path.
% RRT_trials{k}.path_length - holds the length of path `k`.

%% Propagate the Posterior Error Covariance for all RRT paths
% RRT_trials{k}.KF_populated_path - holds the minimal path with all relevant KF
    attributes, specifically Posterior Error Covariance.
terminal_uncertainty = zeros(1, cfg.num_RRT_trials);
for k = 1 : numel(RRT_trials)
    RRT_trials{k}.KF_populated_path = propagate_KF_path(RRT_trials{k}, cfg);
    terminal_uncertainty(k) = RRT_trials{k}.KF_populated_path.final_uncertainty;
end

%% Find minimal terminal uncertainty path
[minimal_uncertainty, minimal_uncertainty_idx] = min(terminal_uncertainty);
minimal_trial = RRT_trials{minimal_uncertainty_idx};
KF_populated_minimal_trial = propagate_KF_path(minimal_trial, cfg);
```

```
plot_path(KF_populated_minimal_trial, "min");
plot_covariance_evolution(KF_populated_minimal_trial, "min");

%% Find maximal terminal uncertainty path
[maximal_uncertainty, maximal_uncertainty_idx] = max(terminal_uncertainty);
maximal_trial = RRT_trials{maximal_uncertainty_idx};
KF_populated_maximal_trial = propagate_KF_path(maximal_trial, cfg);
plot_path(KF_populated_maximal_trial, 'max');
plot_covariance_evolution(KF_populated_maximal_trial, 'max');

%% Find Shortest Path of the trials
[shortest_trial, shortest_trial_idx] = find_shortest_path(RRT_trials);
KF_populated_shortest_trial = propagate_KF_path(shortest_trial, cfg);
plot_path(KF_populated_shortest_trial, 'shortest');
plot_covariance_evolution(KF_populated_shortest_trial, 'shortest');

%% Find best path considering both path length and terminal uncertainty
[best_trial, best_trial_idx, best_path_cost] = find_best_path( ...
    RRT_trials, ...
    terminal_uncertainty, ...
    cfg.best_path_distance_weight, ...
    cfg.best_path_uncertainty_weight);
KF_populated_best_trial = propagate_KF_path(best_trial, cfg);
KF_populated_best_trial.selection_cost = best_path_cost;
plot_path(KF_populated_best_trial, 'best');
plot_covariance_evolution(KF_populated_best_trial, 'best');

%% Add trial-selection summary text box
selection_summary = {
    sprintf('Shortest path found on trial %d of %d', shortest_trial_idx, cfg.
    num_RRT_trials)
    sprintf('Minimal terminal uncertainty path found on trial %d of %d',
    minimal_uncertainty_idx, cfg.num_RRT_trials)
    sprintf('Maximal terminal uncertainty path found on trial %d of %d',
    maximal_uncertainty_idx, cfg.num_RRT_trials)
    sprintf('Best tradeoff path found on trial %d of %d with cost %.3f', best_trial_idx,
    cfg.num_RRT_trials, best_path_cost)
};
annotation('textbox', [0.16 0.76 0.42 0.15], ...
    'String', selection_summary, ...
    'FitBoxToText', 'on', ...
    'BackgroundColor', [1 1 1], ...
    'EdgeColor', [0.25 0.25 0.25], ...
    'FontSize', 10, ...
    'FontWeight', 'bold');
```

simulation_configuation.m

Listing 2: simulation_configuation.m

```
%% simulation_configuation.m
% Author: Matthew Lepis
% Course: ME 656 - Autonomous Navigation for Mobile Robots
```

```
% Assignment: SimLab 2
%
% File: simulation_configuation.m
%
% Description:
%   Builds the configuration struct containing workspace, RRT, sensor, and
%   Kalman filter parameters for the SimLab 2 simulation.

function sim_config = simulation_configuation(obstacle_type, num_RRT_trials,
    plot_RRT_modulo)
[start_state, goal_region, obstacles] = generate_obstacles(obstacle_type);

%
% Robot Constraints
%
%   Robot is a point robot.
%   Robot has 4 laser range finders, one in each cardinal direction - with range: `
    beam_range`

beam_range = 5;

%
% RRT Constraints
%

growth_dist = 2; % meters

%
% KF Constraints
%

time_step = 1; % second

%
% Simulation Constraints
%
x_bounds = [0, 100];
y_bounds = [0, 100];
plot_RRT_runs = false;
plot_RRT_runs_w_ellipses = false;

%
% Best path scoring
%
best_path_distance_weight = 0.5;
best_path_uncertainty_weight = 0.5;

%%
cfg.start_state = start_state;
cfg.goal_region = goal_region;
cfg.obstacles = obstacles;
%
cfg.beam_range = beam_range;
%
```

```
cfg.growth_dist = growth_dist;
%
cfg.time_step = time_step;
%
cfg.num_RRT_trials = num_RRT_trials;
cfg.x_bounds = x_bounds;
cfg.y_bounds = y_bounds;
cfg.plot_RRT_runs = plot_RRT_runs;
cfg.plot_RRT_runs_w_ellipses = plot_RRT_runs_w_ellipses;
cfg.plot_RRT_modulo = plot_RRT_modulo;
cfg.best_path_distance_weight = best_path_distance_weight;
cfg.best_path_uncertainty_weight = best_path_uncertainty_weight;
%
sim_config = cfg;
end
```

RRT.m

Listing 3: RRT.m

```
%% RRT.m
% Author: Matthew Lepis
% Course: ME 656 - Autonomous Navigation for Mobile Robots
% Assignment: SimLab 2
%
% File: RRT.m
%
% Description:
%   Runs repeated Rapidly-exploring Random Tree trials and stores each
%   recovered goal-reaching path with its total path length.

function RRT_trials = RRT(cfg)

function tree = build_RRT(x_pos_init, y_pos_init, edge_color, plot_this_run, cfg)
    tree = Tree(x_pos_init, y_pos_init, cfg);
    tree.extend(edge_color, plot_this_run);
end

function [path, path_length] = reconstruct_goal_path(tree)
    node = tree.vertices(end);  % Goal Node

    reverse_path = [TreeNode.empty(0, 1);];
    path_length = 0;

    while ~node.is_root()
        reverse_path(end+1, 1) = node;
        path_length = path_length + node.cost;
        node = node.parent;
    end
    reverse_path(end+1, 1) = node;
    path = clone_path_nodes(flip(reverse_path));

end
```

```
function path = clone_path_nodes(source_path)
    % Return lightweight path only nodes instead of handles into the full RRT.
    % Otherwise the path root's children can retain the entire tree for each
    % trial.
    path = [TreeNode.empty(0, 1);];

    for node_idx = 1 : numel(source_path)
        state = source_path(node_idx).state;
        path_node = TreeNode(state(1), state(2));
        path_node.idx = node_idx;
        path_node.cost = source_path(node_idx).cost;
        path_node.action = source_path(node_idx).action;

        if node_idx > 1
            path_node.parent = path(node_idx - 1);
        end

        path(end+1, 1) = path_node;
    end
end

function plot_reconstructed_path(path, path_length, path_color)
    path_line_width = 3.5;
    label_offset = [0.75, 0.75];

    if numel(path) < 2
        return;
    end

    for segment_idx = 1 : numel(path) - 1
        plot([path(segment_idx).state(1), path(segment_idx + 1).state(1)], ...
            [path(segment_idx).state(2), path(segment_idx + 1).state(2)], ...
            'Color', path_color, 'LineWidth', path_line_width);
    end

    goal_state = path(end).state;
    text(goal_state(1) + label_offset(1), goal_state(2) + label_offset(2), ...
        sprintf('L = %.2f m', path_length), ...
        'Color', path_color, 'FontSize', 11, 'FontWeight', 'bold', ...
        'HorizontalAlignment', 'left', 'VerticalAlignment', 'bottom');

    drawnow limitrate;
end

function update_progress_bar(completed_runs, total_runs)
    persistent progress_background progress_fill progress_label

    progress_bar_position = [0.20 0.03 0.60 0.025];
    progress_label_position = [0.20 0.055 0.60 0.025];
    progress_fraction = completed_runs / total_runs;

    if isempty(progress_background) || ~isvalid(progress_background)
        progress_background = annotation('rectangle', progress_bar_position, ...
```

```
            'FaceColor', [0.90 0.90 0.90], 'EdgeColor', [0.25 0.25 0.25]);
        progress_fill = annotation('rectangle', ...
            [progress_bar_position(1), progress_bar_position(2), 0, progress_bar_position
    (4)], ...
            'FaceColor', [0.10 0.45 0.80], 'EdgeColor', 'none');
        progress_label = annotation('textbox', progress_label_position, ...
            'String', '', 'EdgeColor', 'none', ...
            'HorizontalAlignment', 'center', 'FontSize', 9);
    end

    progress_fill.Position = [progress_bar_position(1), progress_bar_position(2), ...
        progress_bar_position(3) * progress_fraction, progress_bar_position(4)];
    progress_label.String = sprintf('%d / %d RRT runs complete', completed_runs,
    total_runs);
    drawnow;
end

RRT_trials = cell(1, cfg.num_RRT_trials);
RRT_colors = parula(cfg.num_RRT_trials);
state_state = num2cell(cfg.start_state);
for trial = 1 : cfg.num_RRT_trials
    %% PLOTTING helpers
    %
    update_progress_bar(trial - 1, cfg.num_RRT_trials);
    title(sprintf('RRT run %d of %d', trial, cfg.num_RRT_trials));
    drawnow;
    plot_this_run = cfg.plot_RRT_runs && mod(trial, cfg.plot_RRT_modulo) == 0;

    %% RRT
    %
    tree = build_RRT(state_state{:}, RRT_colors(trial, :), plot_this_run, cfg);
    [path, path_length] = reconstruct_goal_path(tree);

    if plot_this_run
        plot_reconstructed_path(path, path_length, RRT_colors(trial, :));
        if cfg.plot_RRT_runs_w_ellipses
            temp.path = path;
            propagate_KF_path(temp, cfg);
        end
    end

    trial_data = struct();
    trial_data.path = path;
    trial_data.path_length = path_length;

    RRT_trials{trial} = trial_data;


    %% PLOTTING helpers
    %
    update_progress_bar(trial, cfg.num_RRT_trials);
end
title(sprintf('RRT runs complete: %d of %d', cfg.num_RRT_trials, cfg.num_RRT_trials));
end
```

Tree.m

Listing 4: Tree.m

```
%% Tree.m
% Author: Matthew Lepis
% Course: ME 656 - Autonomous Navigation for Mobile Robots
% Assignment: SimLab 2
%
% File: Tree.m
%
% Description:
%   Defines the RRT tree object and helper routines for sampling, nearest
%   neighbor selection, collision-safe extension, and goal detection.

classdef Tree < handle
    properties
        vertices
        c
    end

    methods
        function obj = Tree(x_pos_init, y_pos_init, cfg)
            root = TreeNode(x_pos_init, y_pos_init);
            root.idx = 1;

            obj.vertices = [root];
            obj.c = cfg;
        end

        function extend(obj, edge_color, plot_this_run)
            status = 0;
            while status ~= 1
                x_sample = random_state(obj.c);
                x_near = nearest_neighbor(x_sample, obj);

                [success, x_new, u_new] = new_state(x_sample, x_near, obj.c);
                if success
                    obj.add_node(x_near, x_new, u_new);

                    if plot_this_run
                        plot([x_near.state(1), x_new.state(1)], ...
                            [x_near.state(2), x_new.state(2)], ...
                            'Color', edge_color, 'LineWidth', 0.5);
                        drawnow limitrate;
                    end

                    if collision_check_point(x_new.state(1), x_new.state(2), obj.c.
    goal_region) == 1
                        status = 1;  % Reached Goal
                    else  % x_new == x
```

```
                        status = 0;  % Advanced
                    end
                else
                    status = -1;  % Trapped
                end
            end
        end
    end
    methods (Access = private)
        function obj = add_node(obj, x_near, x_new, u_new)
            x_new.idx = length(obj.vertices) + 1;
            x_new.set_edge(x_near, u_new);

            obj.vertices(end+1, :) = x_new;
        end
    end
end

function nearest_node = nearest_neighbor(x_state, tree)
    nearest_node = tree.vertices(1);
    nearest_distance_sq = sum((nearest_node.state - x_state) .^ 2);

    for node_id = 2 : length(tree.vertices)
        distance_sq = sum((tree.vertices(node_id).state - x_state) .^ 2);
        if distance_sq <= nearest_distance_sq
            nearest_node = tree.vertices(node_id);
            nearest_distance_sq = distance_sq;
        end
    end
end

function [status, x_new, u_new] = new_state(x_state, x_near, c)
    % check if `x` is valid (if it is inside an obstacle or not)
    % if valid:
    %   - status = 1
    %   - calculate the required translation/rotation required to get a linear path with
    length `c.growth_dist` from x_near toward x; set to `u_new`
    %   - make a new node and set far end of the path to `x_new`
    % if not valid:
    %   - status = 0
    %   - x_new = NaN
    %   - u_new = NaN

    x_dir = x_state - x_near.state;
    x_dist_sq = sum(x_dir .^ 2);

    % Check if the sampled node and the nearest neighbor are the same state.
    if x_dist_sq < eps^2
        status = 0;
        x_new = TreeNode.empty(0, 1);
        u_new = [0, 0];
        return;
    end
```

```
    x_dist = sqrt(x_dist_sq);
    x_unit_dir = x_dir / x_dist;
    if (x_dist >= c.growth_dist)
        x_dist = c.growth_dist;
    end

    x_new_state = x_near.state + x_unit_dir * x_dist;

    if collision_check_point(x_new_state(1), x_new_state(2), c.obstacles) == 0 && ...
        collision_check_segment(x_near.state(1), x_near.state(2), x_new_state(1),
    x_new_state(2), c.obstacles) == 0
        x_new = TreeNode(x_new_state(1), x_new_state(2));
        u_new = x_new_state - x_near.state;
        status = 1;
    else
        x_new = TreeNode.empty(0, 1);
        u_new = [0, 0];
        status = 0;
    end
end

function state = random_state(c)
    x_pos_rand = c.x_bounds(1) + (c.x_bounds(2) - c.x_bounds(1)) * rand();
    y_pos_rand = c.y_bounds(1) + (c.y_bounds(2) - c.y_bounds(1)) * rand();
    state = [x_pos_rand, y_pos_rand];
end
```

TreeNode.m

Listing 5: TreeNode.m

```
%% TreeNode.m
% Author: Matthew Lepis
% Course: ME 656 - Autonomous Navigation for Mobile Robots
% Assignment: SimLab 2
%
% File: TreeNode.m
%
% Description:
%   Defines the RRT node object used to store state, parent/child links,
%   edge cost, and the action from its parent node.

classdef TreeNode < handle
    properties
        state
        idx
        parent
        children
        cost
        action
    end

    methods
```

```
        function obj = TreeNode(x, y)
            obj.state = [x, y];
            obj.idx = [];
            obj.parent = [];
            obj.children = TreeNode.empty(0, 1);
            obj.cost = 0;
            obj.action = [];
        end

        function set_edge(obj, parent_node, action)
            obj.parent = parent_node;
            obj.action = action;
            obj.cost = norm(parent_node.state - obj.state);
            parent_node.children(end+1, :) = obj;
        end

        function status = is_root(obj)
            status = isempty(obj.parent);
        end
    end
end
```

propagate_KF_path.m

Listing 6: propagate_KF_path.m

```
%% propagate_KF_path.m
% Author: Matthew Lepis
% Course: ME 656 - Autonomous Navigation for Mobile Robots
% Assignment: SimLab 2
%
% File: propagate_KF_path.m
%
% Description:
%   Propagates Kalman filter covariance along an RRT path and returns the
%   path annotated with state, measurement, covariance, and uncertainty data.

function KF_populated_path = propagate_KF_path(path, cfg)
    %% Copy selected RRT nodes into KF-specific path structs (Performance Step, pruning
    dead data)
    data_pruned_path = initialize_KF_path(path);

    %% Collect measurement data for full path
    sensor_rich_path = collect_sensor_data(data_pruned_path, cfg);

    %% Collect KF variables for the full path
    KF_populated_path = compile_KF_vars(sensor_rich_path, cfg);
end

function kf_path = initialize_KF_path(path)
    rrt_nodes = path.path;
    kf_path = path;
```

```
    empty_kf_node = struct( ...
        'state', [], ...
        'cost', [], ...
        'action', [], ...
        'x_observability', false, ...
        'y_observability', false, ...
        'x', [], ...
        'A', [], ...
        'Q', [], ...
        'H', [], ...
        'R', [], ...
        'P_prev', [], ...
        'P', []);

    kf_nodes = repmat(empty_kf_node, size(rrt_nodes));

    for k = 1 : numel(rrt_nodes)
        kf_nodes(k).state = rrt_nodes(k).state;
        kf_nodes(k).cost = rrt_nodes(k).cost;
        kf_nodes(k).action = rrt_nodes(k).action;
    end

    kf_path.path = kf_nodes;
end

function path = compile_KF_vars(path, cfg)
    nodes = path.path;

    for k = 1 : length(nodes)
        node = nodes(k);

        %% State transition matrix : A
        dt = cfg.time_step;
        node.A = [
            1   0   dt  0;
            0   1   0   dt;
            0   0   1   0;
            0   0   0   1;
        ];

        %% Process noise covariance : Q
        node.Q = eye(4);

        %% State vector : x & Prior Posterior covariance : P_(k-1)
        if k == 1
            node.x = [
                cfg.start_state(1);
                cfg.start_state(2);
                0;
                0;
            ];
            node.P_prev = eye(4);
        else
            node = populate_state_vector(node);
```

```
            node.P_prev = nodes(k - 1).P;
        end

        %% Measurement matrix : H_k
        node.H = build_measurement_matrix(node);

        %% Measurement noise covariance : R_k
        node.R = eye(size(node.H, 1));

        %% Posterior covariance update
        P_predict = node.A * node.P_prev * node.A' + node.Q;
        H = node.H;
        R = node.R;
        node.P = inv(inv(P_predict) + H' * (R \ H));

        nodes(k) = node;
    end

    path.path = nodes;
    path.final_uncertainty = trace(nodes(end).P);
end

function node = populate_state_vector(node)
    x = node.state(1);
    y = node.state(2);

    if isempty(node.action)
        x_vel = 0;
        y_vel = 0;
    else
        x_vel = node.action(1);
        y_vel = node.action(2);
    end

    node.x = [
        x;
        y;
        x_vel;
        y_vel
    ];
end

function H = build_measurement_matrix(node)
    H = zeros(0, 4);

    if node.x_observability
        H(end + 1, :) = [1, 0, 0, 0];
    end
    if node.y_observability
        H(end + 1, :) = [0, 1, 0, 0];
    end

    % Odometry velocity measurements are always available.
    H(end + 1, :) = [0, 0, 1, 0];
```

```
    H(end + 1, :) = [0, 0, 0, 1];
end
```

collect_sensor_data.m

Listing 7: collect_sensor_data.m

```
%% collect_sensor_data.m
% Author: Matthew Lepis
% Course: ME 656 - Autonomous Navigation for Mobile Robots
% Assignment: SimLab 2
%
% File: collect_sensor_data.m
%
% Description:
%   Determines whether each path node has x and/or y observability from
%   nearby range-beam intersections with mapped obstacles.

function path = collect_sensor_data(path, cfg)
    nodes = path.path;
    for k = 1 : length(nodes)
        % Collecting state_vector data
        [x_observable, y_observable] = obstacle_in_reach(nodes(k), cfg);
        nodes(k).x_observability = x_observable;
        nodes(k).y_observability = y_observable;
    end
    path.path = nodes;
end
function [x_observable, y_observable] = obstacle_in_reach(node, cfg)
    obstacles = cfg.obstacles;
    x_observable = 0;
    y_observable = 0;

    x_pos = node.state(1);
    y_pos = node.state(2);

    north_x = x_pos;
    north_y = y_pos + cfg.beam_range;

    west_x = x_pos + cfg.beam_range;
    west_y = y_pos;

    south_x = x_pos;
    south_y = y_pos - cfg.beam_range;

    east_x = x_pos - cfg.beam_range;
    east_y = y_pos;

    if collision_check_segment(x_pos, y_pos, north_x, north_y, obstacles) ...
        || collision_check_point(north_x, north_y, obstacles)
        y_observable = 1;
    end
    if collision_check_segment(x_pos, y_pos, east_x, east_y, obstacles)...
```

```
        || collision_check_point(east_x, east_y, obstacles)
        x_observable = 1;
    end
    if collision_check_segment(x_pos, y_pos,south_x, south_y, obstacles)...
        || collision_check_point(south_x, south_y, obstacles)
        y_observable = 1;
    end
    if collision_check_segment(x_pos, y_pos, west_x, west_y, obstacles)...
        || collision_check_point(west_x, west_y, obstacles)
        x_observable = 1;
    end
end
```

find_shortest_path.m

Listing 8: find_shortest_path.m

```
%% find_shortest_path.m
% Author: Matthew Lepis
% Course: ME 656 - Autonomous Navigation for Mobile Robots
% Assignment: SimLab 2
%
% File: find_shortest_path.m
%
% Description:
%   Finds the minimum-length RRT trial and returns both the selected trial
%   and its trial index.

function [shortest_trial, shortest_path_idx] = find_shortest_path(RRT_trials)
    num_trials = numel(RRT_trials);
    if num_trials == 0
        error('shortest_path:NoTrials', 'RRT_trials must contain at least one trial.');
    end

    path_lengths = zeros(1, num_trials);
    for k = 1 : num_trials
        path_lengths(k) = RRT_trials{k}.path_length;
    end

    [shortest_path_length, shortest_path_idx] = min(path_lengths);

    shortest_trial = RRT_trials{shortest_path_idx};
end
```

find_best_path.m

Listing 9: find_best_path.m

```
%% find_best_path.m
% Author: Matthew Lepis
% Course: ME 656 - Autonomous Navigation for Mobile Robots
% Assignment: SimLab 2
```

```
%
% File: find_best_path.m
%
% Description:
%   Scores each RRT trial by a weighted combination of normalized path
%   length and terminal uncertainty, then returns the lowest-cost trial.

function [best_trial, best_trial_idx, best_cost] = find_best_path(RRT_trials,
    terminal_uncertainty, distance_weight, uncertainty_weight)
    num_trials = numel(RRT_trials);
    if num_trials == 0
        error('best_path:NoTrials', 'RRT_trials must contain at least one trial.');
    end

    if numel(terminal_uncertainty) ~= num_trials
        error('best_path:SizeMismatch', 'terminal_uncertainty must have one value per RRT
     trial.');
    end

    path_lengths = zeros(1, num_trials);
    for k = 1 : num_trials
        path_lengths(k) = RRT_trials{k}.path_length;
    end

    weights = normalize_weights(distance_weight, uncertainty_weight);
    normalized_lengths = normalize_to_unit_interval(path_lengths);
    normalized_uncertainty = normalize_to_unit_interval(terminal_uncertainty);

    combined_cost = weights(1) * normalized_lengths + weights(2) * normalized_uncertainty
    ;
    [best_cost, best_trial_idx] = min(combined_cost);
    best_trial = RRT_trials{best_trial_idx};
end

function weights = normalize_weights(distance_weight, uncertainty_weight)
    weights = [distance_weight, uncertainty_weight];
    if any(weights < 0)
        error('best_path:InvalidWeights', 'Best-path weights must be nonnegative.');
    end

    weight_sum = sum(weights);
    if weight_sum <= 0
        error('best_path:InvalidWeights', 'At least one best-path weight must be positive
    .');
    end

    weights = weights / weight_sum;
end

function normalized_values = normalize_to_unit_interval(values)
    value_range = max(values) - min(values);
    if value_range < eps
        normalized_values = zeros(size(values));
    else
```

```
        normalized_values = (values - min(values)) / value_range;
    end
end
```

plot_path.m

Listing 10: plot_path.m

```
%% plot_path.m
% Author: Matthew Lepis
% Course: ME 656 - Autonomous Navigation for Mobile Robots
% Assignment: SimLab 2
%
% File: plot_path.m
%
% Description:
%   Plots a selected path and labels its selection type, path length, and
%   terminal uncertainty.

function plot_path(trial, type)
    type = lower(char(type));
    path = trial.path;
    path_length = trial.path_length;
    uncertainty = trial.final_uncertainty;
    path_color = selection_color(type);
    path_line_width = 4;
    label_offset = [0.75, -1.50];

    if numel(path) < 2
        return;
    end

    for segment_idx = 1 : numel(path) - 1
        plot([path(segment_idx).state(1), path(segment_idx + 1).state(1)], ...
            [path(segment_idx).state(2), path(segment_idx + 1).state(2)], ...
            'Color', path_color, 'LineWidth', path_line_width);
    end

    goal_state = path(end).state;
    switch type
        case 'max'
            text(goal_state(1) + label_offset(1), goal_state(2) + label_offset(2), ...
                sprintf('Maximal Terminal Uncertainty\n  L = %.2f m\n  Uncertainty = %.2f
    ', path_length, uncertainty), ...
                'Color', path_color, 'FontSize', 11, 'FontWeight', 'bold', ...
                'HorizontalAlignment', 'left', 'VerticalAlignment', 'top');
        case 'min'
            text(goal_state(1) + label_offset(1), goal_state(2) + label_offset(2), ...
                sprintf('Minimal Terminal Uncertainty\n  L = %.2f m\n  Uncertainty = %.2f
    ', path_length, uncertainty), ...
                'Color', path_color, 'FontSize', 11, 'FontWeight', 'bold', ...
                'HorizontalAlignment', 'left', 'VerticalAlignment', 'top');
        case 'shortest'
```

```
            text(goal_state(1) + label_offset(1), goal_state(2) + label_offset(2), ...
                sprintf('Shortest\n  L = %.2f m\n  Uncertainty = %.2f', path_length,
    uncertainty), ...
                'Color', path_color, 'FontSize', 11, 'FontWeight', 'bold', ...
                'HorizontalAlignment', 'left', 'VerticalAlignment', 'top');
        case 'best'
            if isfield(trial, 'selection_cost')
                label = sprintf('Best Tradeoff\n  L = %.2f m\n  Uncertainty = %.2f\n
    Cost = %.3f', ...
                    path_length, uncertainty, trial.selection_cost);
            else
                label = sprintf('Best Tradeoff\n  L = %.2f m\n  Uncertainty = %.2f', ...
                    path_length, uncertainty);
            end
            text(goal_state(1) + label_offset(1), goal_state(2) + label_offset(2), ...
                label, ...
                'Color', path_color, 'FontSize', 11, 'FontWeight', 'bold', ...
                'HorizontalAlignment', 'left', 'VerticalAlignment', 'top');
    end
    drawnow limitrate;
end
```

plot_covariance_evolution.m

Listing 11: plot_covariance_evolution.m

```
%% plot_covariance_evolution.m
% Author: Matthew Lepis
% Course: ME 656 - Autonomous Navigation for Mobile Robots
% Assignment: SimLab 2
%
% File: plot_covariance_evolution.m
%
% Description:
%   Plots one-standard-deviation covariance ellipses along a selected
%   Kalman-filter-annotated path.

function plot_covariance_evolution(trial, type)
    nodes = trial.path;
    if isempty(nodes)
        return;
    end

    hold on;
    ellipse_color = selection_color(type);
    ellipse_line_width = 1.0;
    ellipse_num_points = 60;

    for k = 1 : length(nodes)
        if isempty(nodes(k).P)
            continue;
        end
```

```
        position_covariance = nodes(k).P(1:2, 1:2);
        position_covariance = (position_covariance + position_covariance') / 2;

        [eigenvectors, eigenvalues] = eig(position_covariance);
        eigenvalues = max(real(diag(eigenvalues)), 0);
        [major_variance, major_idx] = max(eigenvalues);
        minor_variance = min(eigenvalues);

        major_axis = sqrt(major_variance);
        minor_axis = sqrt(minor_variance);
        if major_axis < eps && minor_axis < eps
            continue;
        end

        major_direction = eigenvectors(:, major_idx);
        ellipse_angle = atan2(major_direction(2), major_direction(1));

        h = ellipse(major_axis, minor_axis, ellipse_angle, ...
            nodes(k).state(1), nodes(k).state(2), ellipse_color, ellipse_num_points);
        set(h, 'LineWidth', ellipse_line_width);
    end

    drawnow limitrate;
end
```

selection_color.m

Listing 12: selection_color.m

```
%% selection_color.m
% Author: Matthew Lepis
% Course: ME 656 - Autonomous Navigation for Mobile Robots
% Assignment: SimLab 2
%
% File: selection_color.m
%
% Description:
%   Returns consistent RGB colors for selected-path plot categories,
%   including minimum uncertainty, maximum uncertainty, shortest, and best.

function color = selection_color(selection_type)
selection_type = lower(char(selection_type));

switch selection_type
    case 'min'
        color = [0.00, 0.55, 0.55];
    case 'max'
        color = [0.55, 0.25, 0.70];
    case 'shortest'
        color = [0.85, 0.33, 0.10];
    case 'best'
        color = [0.10, 0.45, 0.85];
    otherwise
```

```
        color = [0.20, 0.20, 0.20];
end
end
```

generate_obstacles.m

Listing 13: generate_obstacles.m

```
%% generate_obstacles.m
% Author: Matthew Lepis
% Course: ME 656 - Autonomous Navigation for Mobile Robots
% Assignment: SimLab 2
%
% File: generate_obstacles.m
%
% Description:
%   Creates and plots the SimLab 2 workspace, including the start state,
%   goal region, and selected obstacle layout.

function [start_state, goal_region, obstacles] = generate_obstacles(obstacle_layout)
if nargin < 1
    obstacle_layout = 'original';
end

obstacle_layout = lower(char(obstacle_layout));

close all;
figure(1); hold on;
axis([0 100 0 100]);
box on;

% Define and plot the start state for the planning problem
%               x  y
start_state = [ 5 50];
plot(start_state(1),start_state(2),'.r','MarkerSize',20);

% Define the default goal region for the planning problem.
%              x1 y1  x2  y2   x3  y3  x4  y4
goal_region = [90  0 100   0  100 100  90 100];


switch obstacle_layout
    case 'original'
        % Define the locations of the obstacles.
        %             x1 y1 x2 y2 x3 y3 x4 y4
        obstacles = [  5 10 15 10 15 20  5 20; % obstacle 1
                      10 40 20 40 20 50 10 50; % obstacle 2
                      20 70 30 70 30 80 20 80; % ...etc...
                      30 20 40 20 40 30 30 30;
                      40 50 50 50 50 60 40 60;
                      50  5 60  5 60 15 50 15;
                      55 80 65 80 65 90 55 90;
                      60 40 70 40 70 50 60 50;
```

```
                    70 20 80 20 80 30 70 30
                    75 65 85 65 85 75 75 75 ];
    case 'maze'
        obstacles = [ 15  0 18  0 18 70 15 70;
                    30 30 33 30 33 100 30 100;
                    45  0 48  0 48 65 45 65;
                    60 35 63 35 63 100 60 100;
                    75  0 78  0 78 60 75 60;
                    18 70 28 70 28 73 18 73;
                    33 27 43 27 43 30 33 30;
                    48 65 58 65 58 68 48 68;
                    63 32 73 32 73 35 63 35;
                    78 60 88 60 88 63 78 63 ];
    case 'spiral'
        goal_region = [40 52 54 52 54 54 40 54];
        obstacles = [ 18 18 85 18 85 21 18 21;
                    82 18 85 18 85 82 82 82;
                    18 79 85 79 85 82 18 82;
                    18 34 21 34 21 82 18 82;
                    18 34 72 34 72 37 18 37;
                    69 34 72 34 72 68 69 68;
                    34 65 72 65 72 68 34 68;
                    34 48 37 48 37 68 34 68;
                    34 48 58 48 58 51 34 51;
                    55 48 58 48 58 58 55 58;
                    46 55 58 55 58 58 46 58 ];
    otherwise
        error('Unknown obstacle layout "%s". Use "original", "maze", or "spiral".',
    obstacle_layout);
end

goal_x = [goal_region(1) goal_region(3) goal_region(5) goal_region(7)];
goal_y = [goal_region(2) goal_region(4) goal_region(6) goal_region(8)];
patch(goal_x,goal_y,'green');

num_obstacles = size(obstacles,1);

for i_obs = 1:num_obstacles
    obs_x = [obstacles(i_obs,1) obstacles(i_obs,3) obstacles(i_obs,5) obstacles(i_obs,7)
    ];
    obs_y = [obstacles(i_obs,2) obstacles(i_obs,4) obstacles(i_obs,6) obstacles(i_obs,8)
    ];
    patch(obs_x,obs_y,'blue');
end

end
```

collision_check_point.m

Listing 14: collision_check_point.m

```
%% collision_check_point.m
% Course: ME 656 - Autonomous Navigation for Mobile Robots
```

```
% Assignment: SimLab 2
%
% File: collision_check_point.m
%
% Description:
%   Checks whether a point lies inside any rectangular obstacle or region
%   represented with the SimLab 2 polygon format.

% This function checks whether the point (p_x,p_y) is in collision
% with the set of obstacles described in the matrix 'obstacles',
% which assumes the obstacles are rectangles whose vertices are
% arranged in the sequence used in the script generate_obstacles.m

function collision_found = collision_check_point(p_x,p_y,obstacles)

% returning 1 indicates collision
% returning 0 indicates no collision

collision_found = 0;

num_obstacles = size(obstacles,1);

for i_obs = 1:num_obstacles

    x1 = obstacles(i_obs,1); y1 = obstacles(i_obs,2);
    x2 = obstacles(i_obs,3); y2 = obstacles(i_obs,4);
    x3 = obstacles(i_obs,5); y3 = obstacles(i_obs,6);
    x4 = obstacles(i_obs,7); y4 = obstacles(i_obs,8);

    if (((p_x >= x1) && (p_x <= x2)) && ((p_y >= y1 && p_y <= y3)))
        collision_found = 1;
    end

end
```

collision_check_segment.m

Listing 15: collision_check_segment.m

```
%% collision_check_segment.m
% Course: ME 656 - Autonomous Navigation for Mobile Robots
% Assignment: SimLab 2
%
% File: collision_check_segment.m
%
% Description:
%   Checks whether a line segment intersects any rectangular obstacle in
%   the workspace.

% This function checks whether the segment from (px1,py1) to (px2,py2)
% is in collision with the set of obstacles described in the matrix
% 'obstacles', which assumes the obstacles are rectangles whose vertices
% are arranged in the sequence used in the script generate_obstacles.m
```

```
function collision_found = collision_check_segment(px1,py1,px2,py2,obstacles)

% returning 1 indicates collision
% returning 0 indicates no collision

collision_found = 0;

num_obstacles = size(obstacles,1);

% compute the slope of the line segment
a = px2 - px1;
b = py2 - py1;
m = b/a;

for i_obs = 1:num_obstacles

    x1 = obstacles(i_obs,1); y1 = obstacles(i_obs,2);
    x2 = obstacles(i_obs,3); y2 = obstacles(i_obs,4);
    x3 = obstacles(i_obs,5); y3 = obstacles(i_obs,6);
    x4 = obstacles(i_obs,7); y4 = obstacles(i_obs,8);

    % check segment's y value at x1
    y_at_x1 = m*(x1-px1) + py1;
    t_y_at_x1 = (x1 - px1)/a;
    % check segment's y value at x2
    y_at_x2 = m*(x2-px1) + py1;
    t_y_at_x2 = (x2 - px1)/a;
    % check segment's x value at y1
    x_at_y1 = (y1-py1)/m + px1;
    t_x_at_y1 = (y1 - py1)/b;
    % check segment's x value at y3
    x_at_y3 = (y3-py1)/m + px1;
    t_x_at_y3 = (y3 - py1)/b;

    if ((y_at_x1 >= y1) && (y_at_x1 <= y3) && (t_y_at_x1 >= 0) && (t_y_at_x1 <= 1))
        collision_found = 1; break;
    elseif ((y_at_x2 >= y1) && (y_at_x2 <= y3) && (t_y_at_x2 >= 0) && (t_y_at_x2 <= 1))
        collision_found = 1; break;
    elseif ((x_at_y1 >= x1) && (x_at_y1 <= x2) && (t_x_at_y1 >= 0) && (t_x_at_y1 <= 1))
        collision_found = 1; break;
    elseif ((x_at_y3 >= x1) && (x_at_y3 <= x2) && (t_x_at_y3 >= 0) && (t_x_at_y3 <= 1))
        collision_found = 1; break;
    end

end
```

ellipse.m

Listing 16: ellipse.m

```
%% ellipse.m
% Course: ME 656 - Autonomous Navigation for Mobile Robots
```

```
% Assignment: SimLab 2
%
% File: ellipse.m
%
% Description:
%   Draws covariance ellipses used to visualize positional uncertainty
%   along selected RRT paths.

function h=ellipse(ra,rb,ang,x0,y0,C,Nb)
% Ellipse adds ellipses to the current plot
%
% ELLIPSE(ra,rb,ang,x0,y0) adds an ellipse with semimajor axis of ra,
% a semimajor axis of radius rb, a semimajor axis of ang, centered at
% the point x0,y0.
%
% The length of ra, rb, and ang should be the same.
% If ra is a vector of length L and x0,y0 scalars, L ellipses
% are added at point x0,y0.
% If ra is a scalar and x0,y0 vectors of length M, M ellipse are with the same
% radii are added at the points x0,y0.
% If ra, x0, y0 are vectors of the same length L=M, M ellipses are added.
% If ra is a vector of length L and x0, y0 are  vectors of length
% M~=L, L*M ellipses are added, at each point x0,y0, L ellipses of radius ra.
%
% ELLIPSE(ra,rb,ang,x0,y0,C)
% adds ellipses of color C. C may be a string ('r','b',...) or the RGB value.
% If no color is specified, it makes automatic use of the colors specified by
% the axes ColorOrder property. For several circles C may be a vector.
%
% ELLIPSE(ra,rb,ang,x0,y0,C,Nb), Nb specifies the number of points
% used to draw the ellipse. The default value is 300. Nb may be used
% for each ellipse individually.
%
% h=ELLIPSE(...) returns the handles to the ellipses.
%
% as a sample of how ellipse works, the following produces a red ellipse
% tipped up at a 45 deg axis from the x axis
% ellipse(1,2,pi/8,1,1,'r')
%
% note that if ra=rb, ELLIPSE plots a circle
%

% written by D.G. Long, Brigham Young University, based on the
% CIRCLES.m original
% written by Peter Blattner, Institute of Microtechnology, University of
% Neuchatel, Switzerland, blattner@imt.unine.ch


% Check the number of input arguments

if nargin<1,
  ra=[];
end;
if nargin<2,
```

```
  rb=[];
end;
if nargin<3,
  ang=[];
end;

%if nargin==1,
%  error('Not enough arguments');
%end;

if nargin<5,
  x0=[];
  y0=[];
end;

if nargin<6,
  C=[];
end

if nargin<7,
  Nb=[];
end

% set up the default values

if isempty(ra),ra=1;end;
if isempty(rb),rb=1;end;
if isempty(ang),ang=0;end;
if isempty(x0),x0=0;end;
if isempty(y0),y0=0;end;
if isempty(Nb),Nb=300;end;
if isempty(C),C=get(gca,'colororder');end;

% work on the variable sizes

x0=x0(:);
y0=y0(:);
ra=ra(:);
rb=rb(:);
ang=ang(:);
Nb=Nb(:);

if isstr(C),C=C(:);end;

if length(ra)~=length(rb),
  error('length(ra)~=length(rb)');
end;
if length(x0)~=length(y0),
  error('length(x0)~=length(y0)');
end;

% how many inscribed elllipses are plotted

if length(ra)~=length(x0)
```

```
  maxk=length(ra)*length(x0);
else
  maxk=length(ra);
end;

% drawing loop

for k=1:maxk

  if length(x0)==1
    xpos=x0;
    ypos=y0;
    radm=ra(k);
    radn=rb(k);
    if length(ang)==1
      an=ang;
    else
      an=ang(k);
    end;
  elseif length(ra)==1
    xpos=x0(k);
    ypos=y0(k);
    radm=ra;
    radn=rb;
    an=ang;
  elseif length(x0)==length(ra)
    xpos=x0(k);
    ypos=y0(k);
    radm=ra(k);
    radn=rb(k);
    an=ang(k)
  else
    rada=ra(fix((k-1)/size(x0,1))+1);
    radb=rb(fix((k-1)/size(x0,1))+1);
    an=ang(fix((k-1)/size(x0,1))+1);
    xpos=x0(rem(k-1,size(x0,1))+1);
    ypos=y0(rem(k-1,size(y0,1))+1);
  end;

  co=cos(an);
  si=sin(an);
  the=linspace(0,2*pi,Nb(rem(k-1,size(Nb,1))+1,:)+1);
%  x=radm*cos(the)*co-si*radn*sin(the)+xpos;
%  y=radm*cos(the)*si+co*radn*sin(the)+ypos;
  h(k)=line(radm*cos(the)*co-si*radn*sin(the)+xpos,radm*cos(the)*si+co*radn*sin(the)+ypos
    );
  set(h(k),'color',C(rem(k-1,size(C,1))+1,:));

end;
```

Disclosure of Assistance

Generative AI was used as a supplementary tool to help interpret the assignment prompt and assist with editing portions of this report. The MATLAB implementations, simulation runs, figure generation, and interpretation of the results were produced, verified, and submitted by me, Matthew Lepis.